HDOC Day-69

Generation of Fibonacci Numbers Using Recursion and Counting Recursive Calls in C

Aim

To write a C program using recursion, user-defined functions, conditional statements, and loops to generate the first n Fibonacci numbers and count the total number of function invocations made during execution.

Algorithm

1. Start the program and read a positive integer n.
2. Initialize a global unsigned long long variable callCount to zero.
3. Validate that n is between 1 and 40; otherwise display an error.
4. Define a recursive function fibonacci(n) and increment callCount each time it is entered.
5. If n is 0, return 0; if n is 1, return 1.
6. Otherwise, return fibonacci(n - 1) + fibonacci(n - 2).
7. In main(), call fibonacci(i) for i from 0 through n - 1 and print each term.
8. Display the total number of function invocations and stop.

C Program

```
#include <stdio.h>

/* Global variable to count function calls */
unsigned long long callCount = 0;

/* Function declaration */
unsigned long long fibonacci(int n);

int main()
{
    int n, i;

    printf("Enter the number of terms: ");
    scanf("%d", &n);

    if (n <= 0 || n > 40)
    {
        printf("Invalid input! Enter terms from 1 to 40.\n");
        return 0;
    }

    printf("\nFibonacci Series: ");

    for (i = 0; i < n; i++)
    {
        printf("%llu", fibonacci(i));

        if (i < n - 1)
            printf(" ");
    }

    printf("\nTotal Recursive Calls: %llu\n", callCount);
    return 0;
}

/* Recursive function to generate Fibonacci numbers */
unsigned long long fibonacci(int n)
{
    callCount++;

    if (n == 0)
        return 0;

    if (n == 1)
        return 1;

    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Explanation

The Fibonacci series begins 0, 1, 1, 2, 3, 5, 8, 13, 21, ... Each term after the first two equals the sum of the previous two terms.

Recurrence: F(0) = 0, F(1) = 1, and F(n) = F(n - 1) + F(n - 2) for n >= 2.

Base cases: fibonacci(0) returns 0 and fibonacci(1) returns 1. Otherwise the function makes two recursive calls. The counter is incremented at the start of every invocation, including initial calls from main() and calls that reach a base case.

Recursive call tree for fibonacci(4):

```
F(4)
```

```
|-- F(3)
|   |-- F(2)
|   |   |-- F(1)
|   |   `-- F(0)
|   `-- F(1)
`-- F(2)
    |-- F(1)
    `-- F(0)
```

The call tree for F(4) contains 9 invocations. For the first six terms, the separate calls for F(0) through F(5) require 1 + 1 + 3 + 5 + 9 + 15 = 34 invocations in total.

Recursive Call Count

Term	Value	Calls
F(0)	0	1
F(1)	1	1
F(2)	1	3
F(3)	2	5
F(4)	3	9
F(5)	5	15

Call-count recurrence: C(0) = C(1) = 1; C(n) = 1 + C(n - 1) + C(n - 2). The naive recursive approach has exponential time complexity O(phi^n), where phi is approximately 1.618, and O(n) auxiliary stack space.

Test Case Table

Case	Number of terms	Expected series	Total calls
1	1	0	1
2	2	0 1	2
3	3	0 1 1	5
4	4	0 1 1 2	10
5	5	0 1 1 2 3	19
6	6	0 1 1 2 3 5	34
7	7	0 1 1 2 3 5 8	59
8	8	0 1 1 2 3 5 8 13	100
9	0	Invalid input	-
10	-5	Invalid input	-

Sample Outputs

Six Fibonacci Terms

```
Enter the number of terms: 6

Fibonacci Series: 0 1 1 2 3 5
Total Recursive Calls: 34
```

Five Fibonacci Terms

```
Enter the number of terms: 5

Fibonacci Series: 0 1 1 2 3
Total Recursive Calls: 19
```

Single Fibonacci Term

```
Enter the number of terms: 1

Fibonacci Series: 0
Total Recursive Calls: 1
```

Eight Fibonacci Terms

```
Enter the number of terms: 8

Fibonacci Series: 0 1 1 2 3 5 8 13
Total Recursive Calls: 100
```

Invalid Input

```
Enter the number of terms: 0
Invalid input! Enter terms from 1 to 40.
```

Result

The C program uses recursion, a user-defined fibonacci() function, a global call counter, loops, and conditional statements to generate the first n Fibonacci numbers. It counts all function invocations, including base cases and initial calls, and validates the number of requested terms.